Entrega 3 · Documento 3

# Análisis de captura de tráfico

Captura y análisis del tráfico de la aplicación web por caso de uso

Sistema integrado de eventos laborales (JML)

Andina Servicios S.A.S. (empresa ficticia · datos ficticios)

Politécnico Grancolombiano · Gerencia de Proyectos en Seguridad de la Información · Grupo D01

24 de octubre de 2026

Todas las pruebas se ejecutaron contra el despliegue local del sistema (BunkerWeb + aplicación + PostgreSQL + GLPI 11 + MariaDB en Docker Compose). No se ejecutaron pruebas activas contra servicios de terceros (Supabase). Los secretos y tokens de las evidencias están enmascarados.

# 1. Resumen

Se capturó el tráfico de red mientras se ejecutaban los casos de uso del Documento 1, en tres puntos del sistema, y se analizó con tshark. El objetivo es comprobar, sobre el cable, tres propiedades de seguridad de la información: que el tráfico externo viaja cifrado (confidencialidad), que los campos restringidos no se filtran a donde no deben (confidencialidad), y qué protocolos y datos circulan por cada tramo interno (base para la propuesta de controles del Documento 4).

| Punto de captura | Tramo observado | Archivo | Tramas |
|---|---|---|---|
| Contenedor bunkerweb | Cliente↔WAF (externo) y WAF↔web (interno) | bunkerweb.pcap | 3 264 |
| Contenedor postgres | Aplicación/worker ↔ PostgreSQL | postgres.pcap | 6 537 |
| Contenedor glpi | Worker↔API GLPI y GLPI↔MariaDB | glpi.pcap | 15 093 |

Captura con `tcpdump` (imagen netshoot) adosada al espacio de red de cada contenedor. Análisis con `tshark` (`scripts/analizar-trafico.sh`). Los valores de token, cookie y clave se enmascararon en los extractos; los archivos .pcap crudos se conservan en `evidencias/trafico/`.

## 2. Topología de los tramos

El cliente llega por HTTPS al WAF (10.20.40.2). BunkerWeb termina el TLS y reenvía en claro por HTTP a la aplicación (web:8000) o a GLPI (glpi:80), dentro de la red `app`. La aplicación habla con PostgreSQL por el protocolo nativo (5432); el worker habla con la API REST de GLPI por HTTP; GLPI habla con MariaDB (3306). Las redes `app`, `pg_net` y `db_net` son internas de Docker; `db_net` es `internal:true`.

## 3. Jerarquía de protocolos por captura

Distribución de tramas por protocolo (`tshark -z io,phs`). En el borde predomina TLS (tráfico cifrado); hacia dentro aparece HTTP en claro, el protocolo de PostgreSQL y el de MySQL/MariaDB, cada uno en su red.

| Captura | Protocolos principales (tramas) |
|---|---|
| bunkerweb.pcap | TLS 758 (cifrado, externo) · HTTP 252 (claro, interno) · DNS 10 |
| postgres.pcap | PostgreSQL 5 650 · TCP de control; sin TLS (red interna de desarrollo) |
| glpi.pcap | MySQL/MariaDB 11 291 · HTTP 276 (API REST de GLPI, JSON) |

Fuentes: `analisis/bunkerweb-protocolos.txt`, `postgres-protocolos.txt`, `glpi-protocolos.txt`.

## 4. Tramo externo: cliente ↔ WAF (confidencialidad)

El tramo cliente↔WAF es TLS 1.3. El `ClientHello` ofrece TLS 1.3 (0x0304) y 1.2 (0x0303); el `ServerHello` fija la suite 0x1302 (TLS_AES_256_GCM_SHA384). A partir de ahí todo es `Application Data` cifrado. El puerto 80 responde 301 a HTTPS.

**tshark: ClientHello y datos de aplicación cifrados (tramo externo, 10.20.40.1 ↔ 10.20.40.2:8443)**

```
# ClientHello: versiones ofrecidas y ALPN
11.215740 10.20.40.1 app.localhost 0x0304,0x0303 h2,http/1.1
# ServerHello: version y suite elegidas
0x0304 0x1302 (TLS_AES_256_GCM_SHA384) x48 conexiones
# Application Data: contenido cifrado (type 23), ilegible en el cable
frame 26 10.20.40.2 -> 10.20.40.1 tls.app_data: 836e72d7d436bdc1237bc9dcf8787aaec3bcf6da...
frame 28 10.20.40.1 -> 10.20.40.2 tls.app_data: 0b7dcc0e7cfba113431c9a47b47d80aa1bac92ee...
# HTTP sobre el puerto 80 -> redirección forzada a HTTPS
10.20.40.1 GET / -> 10.20.40.2 301 https://app.localhost/
```

Confirma RS-01 (TLS en el tráfico externo) y la amenaza T19: el único punto de entrada es el WAF por TLS. Fuentes: `tls-clienthello.txt`, `tls-serverhello.txt`, `tls-datos-cifrados.txt`, `http-80-redireccion.txt`.

## 5. Confidencialidad de los campos restringidos

La prueba clave de confidencialidad: se siguió el rastro de dos valores centinela, el motivo de un retiro (`CENTINELA-MOTIVO-RET-7f3a`) y la descripción de una emergencia (`CENTINELA-DESCRIPCION-EMG-9c1b`), por los tres puntos de captura. El resultado respeta el invariante 5 del sistema: el campo restringido vive solo en PostgreSQL y **nunca llega a GLPI**.

| Tramo / captura | ¿Aparece el centinela? | Interpretación |
|---|---|---|
| Cliente↔WAF, externo (TLS) | No (cifrado) | Viaja dentro del TLS; ilegible en el cable |
| WAF↔web, interno (HTTP) | Sí, en el cuerpo POST | Red app interna; el dato entra al servidor |
| Aplicación↔PostgreSQL | Sí (57 tramas) | Se almacena en solicitud_restringida (correcto) |
| Worker↔API GLPI | No (0) | El generador de tickets lo excluye (RF-20, invariante 5) |

**tshark: el centinela del retiro llega a PostgreSQL pero no a GLPI**

```
# postgres.pcap: el valor viaja a PostgreSQL (almacenamiento legítimo en solicitud_restringida)
frame 3041 172.18.0.3 -> 172.18.0.2 Simple query (contiene CENTINELA-MOTIVO-RET-7f3a)
recuento en postgres.pcap: 57 tramas con 'CENTINELA'
# glpi.pcap: el valor NO aparece en ningún tramo hacia GLPI
recuento en glpi.pcap: 0 tramas con 'CENTINELA'
# bunkerweb.pcap: solo en el tramo interno HTTP (10.20.40.2 -> 10.20.40.3:8000), nunca en el TLS externo
frame 618 10.20.40.2 -> 10.20.40.3:8000 (cuerpo POST, tramo interno en claro)
```

Que el centinela aparezca en el tramo interno WAF↔web en claro es esperado (el WAF termina el TLS); ese tramo vive en la red Docker app, no expuesta. Lo relevante es que fuera del host viaja cifrado y que a GLPI no llega. Fuentes: `pg-centinela.txt`, `glpi-centinela.txt`, `borde-centinela.txt`.

## 6. Capturas por caso de uso (tramo interno HTTP)

Durante el recorrido, el tramo WAF↔web registró 95 peticiones HTTP. La tabla resume las peticiones observadas y el caso de uso que las originó. El método, la ruta y el código coinciden con los del Documento 1, vistos ahora desde la red.

| Peticiones (método y ruta) | Caso de uso | Observado en la captura |
|---|---|---|
| POST /ingreso (× 4) | CU-01 | Cuerpo con correo/contraseña; respuesta sin token en claro |
| POST /solicitudes/nueva/VAC (× 4) | CU-02, CU-03 | Registro y rechazo del duplicado |
| POST /solicitudes/nueva/CDP (× 2) | CU-04 | Registro del jefe |
| POST /solicitudes/SOL-…/aprobar (× 2) | CU-05 | Decisión de RRHH |
| POST /solicitudes/nueva/ING (× 4) | CU-06 | Multipart con el PDF; rechazo del no-PDF |
| POST /solicitudes/nueva/RET, EMG (× 4) | CU-07, CU-08 | Emisión con campo restringido |
| GET /dashboard, /cuentas, /colaboradores | CU-09 | Lectura de técnicos |
| GET/POST /auditoria (× 5) | CU-11 | Consulta y verificación de la cadena |
| GET /administracion/… (× 2) | CU-12 | Área de administración |
| POST /salir (× 1) | CU-13 | Cierre de sesión |

Las cookies de respuesta observadas llevan los atributos `HttpOnly`, `Secure`, `SameSite=Lax` y `Max-Age=28800` (8 h), y la cabecera `Content-Security-Policy` estricta sale del upstream. Fuentes: `interno-web-peticiones.txt`, `interno-web-cabeceras.txt`.

## 7. Integridad: consultas parametrizadas a la base

El tráfico a PostgreSQL muestra sentencias con identificadores entre comillas y valores ligados, propias del ORM de Django, dentro de transacciones (`BEGIN` … `COMMIT`): 241 BEGIN y 235 COMMIT en la ventana. No se observan sentencias construidas por concatenación de input. Respalda RS-05 y la defensa frente a la amenaza T4 (inyección) comprobada en el Documento 2.

**tshark: consultas del ORM a PostgreSQL (identificadores citados, dentro de transacción)**

```
241 x BEGIN
235 x COMMIT
135 x SELECT "auth_user"."id", "auth_user"."password", ... FROM "auth_user" WHERE ...
66 x SELECT "auth_group"."name" FROM "auth_group" INNER JOIN "auth_user_groups" ON ...
25 x SELECT "consecutivo"."clave", "consecutivo"."ultimo" FROM "consecutivo" WHERE ... FOR UPDATE
```

La lectura de roles en cada petición (`auth_group` / `auth_user_groups`) confirma que la autorización se evalúa contra la base en cada acceso, no contra la sesión. Fuente: `pg-consultas.txt`.

## 8. Tramo interno worker ↔ API de GLPI

El worker crea y consulta tickets por la API REST de GLPI (`apirest.php`), en JSON sobre HTTP, dentro de la red `app`. La autenticación usa `App-Token` y `Session-Token` en cabeceras. Esos tokens viajan en claro en este tramo interno (observación C-01 del Documento 4). Nunca salen del host: la API de GLPI está bloqueada en el WAF desde el exterior (verificado en el Documento 2).

**tshark: llamadas del worker a la API de GLPI (tokens enmascarados)**

```
10.20.40.5 GET /apirest.php/initSession App-Token: <oculto> Authorization: user_token <oculto>
10.20.40.5 POST /apirest.php/Ticket App-Token: <oculto> Session-Token: <oculto>
10.20.40.5 GET /apirest.php/Ticket?...externalid=SOL-... (idempotencia por referencia externa)
10.20.40.5 POST /apirest.php/Document (subida del soporte PDF, una sola vez)
recuento de cabeceras con credenciales en claro: App-Token 130, Session-Token 129, user_token 1
```

Fuentes: `glpi-api-peticiones.txt`, `glpi-api-credenciales-en-claro.txt`.

## 9. Observaciones de seguridad del tráfico

| C-01 · Tokens de la API de GLPI en claro en el tramo interno | Baja |
|---|---|
| **Evidencia** | glpi.pcap: cabeceras App-Token/Session-Token en HTTP sin TLS en el tramo worker↔GLPI (glpi-api-credenciales-en-claro.txt). |
| **Impacto** | Bajo: el tramo vive en la red Docker interna app; la API de GLPI no es accesible desde el exterior (403 en el WAF). Un atacante necesitaría ya estar dentro de la red de contenedores. |
| **Corrección** | Defensa en profundidad: TLS en el tramo worker↔GLPI, o una malla mTLS, si el modelo de amenaza incluye un atacante con acceso a la red interna. |
| **Falsabilidad** | Un tshark en bunkerweb.pcap que mostrara App-Token hacia el exterior indicaría que la API de GLPI quedó expuesta. |

| C-02 · Tramos internos de base de datos sin TLS (entorno local) | Informativa |
|---|---|
| **Evidencia** | postgres.pcap y glpi.pcap: protocolo de PostgreSQL y de MariaDB sin TLS en las redes internas. |
| **Impacto** | Solo en el despliegue local. En producción, la aplicación se conecta a Supabase con TLS verify-full; MariaDB solo es accesible por db_net (internal:true). |
| **Corrección** | Ninguna en producción. Documentar que las capturas de BD son del entorno local y no representan el tramo cifrado a Supabase. |
| **Falsabilidad** | Una conexión a Supabase sin TLS en los ajustes de producción sería la regresión. |

## 10. Conclusión

El análisis de tráfico confirma las tres propiedades buscadas: el tráfico externo viaja cifrado con TLS 1.3; el campo restringido nunca abandona el host en claro ni llega a GLPI; y las consultas a la base están parametrizadas y envueltas en transacciones. Las dos observaciones (C-01 y C-02) son de severidad baja o informativa y se refieren a los tramos internos, protegidos por el aislamiento de red. Alimentan la propuesta de controles del Documento 4.